

Team Name: team_16

Phase 0: Connect & Baseline Latency Report

Repository: [https://github.com/arabbit20131208-hue/hft-cpp_team16](https://github.com/arabbit20131208-hue/hft-cpp_team16)

Target Platform: macOS / C++ (hft_bot)

1. Baseline Latency Metrics

Below are the official offline baseline tick-to-trade latency measurements generated via python3 scripts/latency_replay.py --self-test --cmd "hft/cpp_client/build/hft_bot --replay".

Metric	Value
Snapshots Evaluated	2,000 (Warmup: 50, Timed: 1,950)
Throughput	22,540 snapshots/sec (over 0.089s wall clock)
Minimum Latency (min)	23.71 μs
Median Latency (p50)	31.75 μs
Mean Latency	30.83 μs
p90 Latency	33.17 μs
p99 Latency	38.58 μs
Tail Latency (p99.9)	55.51 μs
Max Latency	60.75 μs

Latency Distribution Histogram

2. Verification Proofs

[illegible]

```
(base) hangsecho@Mac hft-cpp_team16 % ./run_bot.sh
connecting as team_16_trader_1 to wss://feed.algoarenafin.duckdns.org
[main] AlgoArena HFT bot - team=team_16_trader_1 url=wss://feed.algoarenafin.duckdns.org
[arena] connected to wss://feed.algoarenafin.duckdns.org as team_16_trader_1 (trader)
[session] FEE_SCHEDULE - Venue fee schedule: taker 30.0 bps, maker rebate 5.0 bps
[session] SESSION_OPEN - Session already open - you joined mid-session.
LAT AMZN 164
[fill] taker sell 1 AMZN @ 172.68 pos=-1
LAT TSLA 186
LAT AMZN 260
LAT NFLX 103
[fill] taker buy 1 TSLA @ 321 pos=1
[fill] taker buy 1 AMZN @ 173.82 pos=0
[fill] taker buy 1 NFLX @ 670.1 pos=1
LAT NFLX 160
[fill] taker buy 1 NFLX @ 670.49 pos=2
LAT NFLX 164
[fill] taker buy 1 NFLX @ 670.93 pos=3
LAT AMD 142
[fill] taker buy 1 AMD @ 242.57 pos=1
```

Screenshot 3: Bot on the LATENCY Board

ALGOARENA MARKET SIMULATION MARKET FLOW STATS LATENCY QUEUE SEASON 00:00 TICK #301600 OPEN PLAY connected READ ONLY SIGN IN													
STANDINGS 🏠 House Bots +109.4% 📈 Heights Frequency Trading +75.0% 👤 team6 +73.7% 🥥 Coconut +60.7% 🏢 The Exchange +56.6% 👤 SHADALOO +34.3% 👤 TEAM 16 +23.1% 👤 Seb Alpha +21.0% 👤 Hyperion +15.0%													
REACTION LATENCY - server-measured time from a market update to each bot's next order. Ranked by p99.9 (the tail that wins races). Lower is faster. Unfakeable! timed on the exchange. Fastest: shadalo0_trader_1 at 75.1ms p99.9 - median 332.8ms - slowest 10151.2ms - 135x spread front-to-back													
#	PARTICIPANT	ROLE	p50	p99	p99.9	SAMPLES	PASSIVE %	REBATES	MARKOUT 1s	MM SCORE	OTR	TAIL (p99)	
1	shadalo0_trader_1	trader	13.3ms	30.0ms	75.1ms	2000	0%	\$0.43	\$-0.64	\$-2005.96	1.0		
2	team6_trader_1	trader	15.0ms	36.9ms	76.9ms	644	0%	\$0.00	+\$0.00	\$-442.34	1.0		
3	coconut_trader_1	trader	31.2ms	133.6ms	133.7ms	81	0%	\$0.00	+\$0.00	+\$4.78	4.2		
4	team16_trader_1	trader	36.4ms	207.1ms	222.1ms	105	1%	\$0.12	\$-0.16	\$-53.34	1.0		
5	the_exchange_trader_1	trader	17.3ms	150.3ms	230.3ms	2000	0%	\$1.35	\$-2.25	\$-1510.53	1.0		
6	team24_trader_1	trader	122.3ms	301.9ms	314.6ms	20	-	\$0.00	+\$0.00	+\$0.00	40.0		
7	hyperion_trader_1	trader	15.0ms	138.3ms	332.8ms	1309	46%	\$5.12	\$-5.46	\$-47.00	32.0		
8	heights_frequency_trading_trader_1	trader	42.6ms	494.3ms	516.9ms	2000	4%	\$10.25	\$-14.59	+\$3573.43	1.0		
9	house_bots_broker	broker	232.6ms	516.6ms	520.5ms	2000	100%	\$64539.42	\$-33351.71	+\$585310.21	19.9		
10	flow_vwap_1	trader	254.6ms	515.2ms	549.7ms	2000	1%	\$164.16	\$-141.69	\$-305151.37	1.4		
11	flow_retail_1	trader	261.3ms	515.8ms	2678.9ms	2000	1%	\$127.83	\$-145.83	\$-119791.27	2.2		
12	flow_retail_2	trader	257.1ms	518.2ms	10151.2ms	2000	1%	\$136.04	\$-150.50	\$-118939.18	2.2		

3. Performance Analysis: Where Time Goes and Why

In Phase 0, the baseline client achieves a median tick-to-trade latency of $31.75 \mu s$ and a tail latency (p99.9) of $55.51 \mu s$. The core time consumption during standard execution (p50) is dominated by stream I/O operations (reading from stdin/writing to stdout in replay mode or WebSocket framing in live mode), string allocations during JSON message deserialization, and the evaluation of order book state within the `on_book()` handler. The significant increase at the tail latency (p99.9 / max spiking up to $60.75 \mu s$) is primarily driven by OS-level execution jitter, CPU context switching, dynamic memory reallocations inside standard container types (e.g., `std::unordered_map` bucket management), and buffer flushing during high-throughput bursts of market data snapshots. The p99.9 latency is approximately 1.75 times the p50 latency, suggesting that the tail is caused by occasional overhead rather than a consistently slow code path.